

CMS-R4 • Sistema Plugin

Questa guida riassume i **file coinvolti**, il **flusso di installazione**, lo **schema del manifest**, come **caricare gli asset** (admin/front) e come **scrivere un plugin** (con esempi). In fondo trovi checklist e troubleshooting.

1) Mappa dei file e responsabilità

Backend (installazione/registro)

- `app/Http/Controllers/Admin/PluginController.php`
Endpoint admin per: elenco, upload ZIP, abilita/disabilita, rimozione.
- `app/Services/PluginManager.php`
Cuore del sistema: estrae ZIP, valida e sposta i file, copia gli asset pubblici, calcola URL versionati, espone liste asset e registry blocchi.
- `app/Models/Plugin.php`
Tabella `plugins` con campi `slug, name, version, author, manifest, enabled` (cast `manifest=>array, enabled=>bool`).

Frontend pubblico

- `resources/views/layouts/app.blade.php` (**Template pubblico**)
Imposta SEO/branding, include Bootstrap e (facoltativamente) **asset front dei plugin** (vedi §4).
- `resources/views/pages/show.blade.php`
Recupera gli asset front dai plugin e li inserisce negli stack `@push('styles')/ @push('scripts')`.
- `resources/views/partials/page_renderer.blade.php`
Renderer universale dei blocchi. Per i blocchi `type` che iniziano con `plugin:` emette un **placeholder** `.pb-plugin` e un JS che tenta il **mount** cercando `window.BuilderPlugins[<type>]` (`mount()` o `renderView()`).

Builder (Admin > Pagine > Edit)

- `resources/views/admin/pages/edit.blade.php`
Editor visuale. Riceve `window.__PLUGIN_BLOCKS__` (da `PluginManager::buildBlocksRegistry()`), offre API JS che i plugin possono usare in admin (es. `renderEditor(sec, blk)`).

2) Configurazione (consigliata)

Crea `config/plugins.php` per centralizzare i path (le chiavi sono già lette dal `PluginManager`).

```
return [  
    'path'          => base_path('plugins'),           // sorgente plugin  
    'public_path'   => public_path('plugins'),          // /public/plugins
```

```

        'web_path'      => '/plugins',           // URL base pubblico
        'manifest'     => 'plugin.json',        // nome manifest
        'public_dir'   => 'public',             // cartella asset nel
plugin
        'admin_entry'  => 'admin.js',           // entry JS admin
(opzionale)
        'view_entry'   => 'view.js',           // entry JS front
(fallback)
    ];

```

Il `PluginManager` usa `ensureDirs()` per creare `path` e `public_path` se mancanti.

3) Flusso di installazione (upload ZIP)

1. **Upload** da `PluginController@upload` (`/admin/plugins/upload`): valida MIME e dimensioni.
2. **Estrazione** in una cartella temporanea.
3. **Ricerca** del `plugin.json` (case-insensitive) ovunque nello ZIP.
4. **Validazione minima**: `slug` (`[a-z0-9-]+`) + lettura di `name/version/author/blocks/assets`.
5. **Spostamento** cartella radice del plugin in `config('plugins.path')/<slug>`.
6. **Copia** degli asset pubblici: `<plugin>/public/*` → `/public/plugins/<slug>/*`.
7. **Upsert** nel DB (`plugins`): salva anche il `manifest` (cast array) e imposta `enabled=false` di default (opzionale, a tua scelta).

Rimozione: `destroy()` cancella cartella sorgente e pubblica, poi elimina il record.

4) Caricamento degli asset (front & admin)

Il manager espone più metodi:

- `adminAssets(): ['css'=>[], 'js'=>[]]`
Include (1) `admin_entry` se esiste e (2) `assets.front` (se dichiarati) — in ordine corretto e **deduplicati**. Usalo nelle viste **admin**.
- `frontAssets(): array`
Ritorna un **array piatto** di URL (CSS/JS) dichiarati in `assets.front` per i plugin abilitati (con versioning `?v=mtime`).
- `frontendAssets(): array`
Fallback che forza l'inclusione del solo `view_entry` (es. `view.js`) se presente.

Evitare doppi include

Nella tua base ci sono **due** punti che includono asset front: - in **layout pubblico** (in fondo al `<body>`) e - in `pages/show.blade.php` via `@push('styles')/@push('scripts')`.

Scegli una strategia: - **A. Globale (semplice):** lascia l'include nel **layout** e rimuovi quello in `show.blade.php`.

✓ sempre disponibili, ✗ possibile caricare asset inutili in pagine che non usano plugin. - **B. Per-pagina (ottimizzata):** togli l'include dal **layout** e mantieni `show.blade.php`.

✓ carichi solo quando necessario, ✗ devi ricordarti di farlo nelle viste che usano plugin.

Consiglio: adotta **B** (per-pagina). In `layouts/app.blade.php` rimuovi il blocco che itera `$pluginFrontAssets`. In `show.blade.php` mantieni la logica che costruisce `$pf_css/$pf_js`.

`type="module"`

Gli script vengono inclusi con `defer` **senza** `type="module"`. **Compila i plugin** come IIFE che scrivono su `window`, non come moduli ES.

5) Manifest (`plugin.json`) — schema e esempio

Lo **schema minimo** accettato dal manager:

```
{
  "slug": "r4-logos-carousel",
  "name": "R4 Carosello Loghi",
  "version": "1.1.0",
  "author": "R4Software",
  "blocks": [
    { "type": "plugin:r4-logos-carousel", "label": "Carosello Loghi",
  "icon": "bi-sliders", "add_columns": 12 }
  ],
  "assets": {
    "front": ["view.js", "view.css"]
  },
  "admin_entry": "admin.js"
}
```

Campi chiave: - `slug` — unico, `[a-z0-9-]+` (usato per le cartelle e per i path pubblici). - `blocks[]` — definisce i **blocchi del Builder**. `type` **deve** iniziare con `plugin:`. - `assets.front[]` — path relativi alla cartella `public/` del plugin (CSS/JS). - `admin_entry` — (opz.) JS caricato nell'editor (admin). - `view_entry` — (opz.) usato da `frontendAssets()` come fallback.

Il manager è tollerante: `manifest` può essere array/JSON; `assets.front` può contenere sia CSS sia JS.

6) Struttura cartelle del plugin

```
plugins/  
└─ r4-logos-carousel/          (sorgente, spostato qui dopo  
l'installazione)  
    └─ plugin.json  
    └─ public/                  ( tutto ciò che va in /public/  
plugins/<slug>/  
    └─ view.js  
    └─ view.css  
    └─ admin.js                 (se serve all'editor)  
    └─ (altre cartelle interne a tua scelta)
```

Dopo l'installazione, i contenuti di `public/` vengono copiati in:

```
/public/plugins/<slug>/*
```

Accessibili via URL:

```
/plugins/<slug>/view.js?v=123456789
```

7) Integrazione a runtime (public & admin)

Public renderer (`page_renderer.blade.php`)

- Per un blocco con `type: 'plugin:...'` genera un div:

```
<div class="pb-plugin" data-type="plugin:r4-logos-carousel" data-block-id="..." data-payload='{ "...": ... }'>  
  <div class="pb-plugin--placeholder">Plugin in caricamento...</div>  
</div>
```

- Un loop JS tenta di montare: cerca in `window.BuilderPlugins[type]` una delle API:
- `mount(el, data)` – riceve il placeholder e i dati del blocco; sostituisci o popola l'interno.
- `renderView(data): string` – fallback: ritorna HTML stringa.
- Eventi utili: `document.dispatchEvent(new Event('r4lc:refresh'))` forza un nuovo sweep; all'init dei plugin emetti `plugins:ready`.

Admin builder (`edit.blade.php`)

- `window.__PLUGIN_BLOCKS__` elenca i blocchi disponibili (dal `PluginManager::buildBlocksRegistry()`).
- Se un plugin definisce `renderEditor(sec, block)`, l'editor lo usa nel pannello impostazioni per un'esperienza nativa.
- Altrimenti mostra un placeholder + eventuale `renderView()` come preview.

8) Come scrivere un plugin (linee guida)

8.1. JavaScript (IIFE, no ES modules)

public/view.js

```
(function(){
  window.BuilderPlugins = window.BuilderPlugins || {};

  // Nome tipo dal manifest
  const TYPE = 'plugin:r4-logos-carousel';

  // Montaggio sul frontend pubblico
  function mount(el, data){
    // data è il payload salvato nel blocco (admin)
    const logos = Array.isArray(data.logos) ? data.logos : [];
    const gap = data.gap || 24;
    el.innerHTML = `
      <div class="r4lc" style="--r4lc-gap:${gap}px">
        <div class="r4lc-track">${logos.map(src => ``).join('')}</div>
      </div>
    `;
  }

  // Editor opzionale (usato in admin)
  function renderEditor(sec, block){
    const b = block.data || (block.data = {});
    b.logos = Array.isArray(b.logos) ? b.logos : [];
    return `
      <div class="border p-2 rounded bg-light">
        <div class="small text-muted mb-2">Carica loghi e imposta il gap</div>
        <div class="d-flex gap-2 flex-wrap">
          <button type="button" class="btn btn-sm btn-outline-primary"
            onclick="pickGallery('${sec.id}', '${
block.id}', 'gallery')">Aggiungi loghi</button>
          <input type="number" class="form-control form-control-sm w-auto"
min="0" step="2"
            value="${b.gap||24}" oninput="updatePluginField('${
sec.id}', '${block.id}', 'data.gap', parseInt(this.value||0,10))">
          </div>
        </div>`;
  }

  window.BuilderPlugins[TYPE] = { mount, renderEditor };
  document.dispatchEvent(new Event('plugins:ready'));
})();
```

`public/view.css` (minimo per evitare colonne verticali):

```
.r4lc{ overflow:hidden }  
.r4lc-track{ display:flex; align-items:center; gap:var(--r4lc-gap,24px); }  
.r4lc-track img{ height:var(--r4lc-item-h,72px); width:auto; object-fit:contain; display:block }
```

Nota: niente `type=module`. Lo script deve allegarsi a `window.BuilderPlugins`.

8.2. Manifest

Vedi §5. Assicurati che `blocks[].type` inizi con `plugin:` e coincida con il tipo usato in JS.

8.3. CSS: scoping

Prefissa le classi (es. `.r4lc`), evita collisioni globali, non sovrascrivere Bootstrap.

8.4. Dati del blocco

Nel builder, i plugin scrivono in `block.data.*` tramite `updatePluginField(sid,bid,'data.foo', value)`.

9) Linea guida d'uso (per redattori/admin)

1. Vai in **Amministrazione** → **Plugin**.
2. **Carica** il file `.zip` del plugin.
3. Nell'elenco, **Abilita** il plugin.
4. Vai su **Pagine** → **Modifica** e aggiungi un **Blocco** → scegli il blocco del plugin dal **palette** (icona puzzle).
5. Configura i campi dal pannello impostazioni (o dall'editor nativo del plugin se fornito).
6. **Salva / Pubblica** la pagina. In front il renderer monterà automaticamente i plugin dopo che i relativi asset saranno caricati.

10) Troubleshooting

- **"Plugin in caricamento..." resta fisso**
- Verifica in DevTools che `/plugins/<slug>/view.js` e `view.css` vengano caricati (200 OK).
- Controlla che lo script registri `window.BuilderPlugins['plugin:...']` e che **NON** sia un modulo ES.
- L'evento `plugins:ready` può forzare un nuovo mount.
- **Stili non applicati / loghi in verticale**
- Assicurati che `view.css` sia incluso (vedi §4 strategia asset).

- Minimo richiesto: `display: flex` sulla track (vedi snippet §8.1).
 - **Doppi asset**
 - Allinea alla **strategia B**: rimuovi l'include globale del layout e lascia la gestione in `show.blade.php`.
 - **Variante immagini errata (thumb croppato)**
 - In admin usa `image.quality` o costruisci URL con suffisso `_{thumb|25|59|75|full}` coerente; il renderer ha `pb_variant_url()`.
 - **ZIP non riconosciuto**
 - Il manifest **deve** chiamarsi come `config('plugins.manifest')` (default `plugin.json`) ed essere presente **da qualche parte** nello ZIP.
-

11) API riassuntiva (PluginManager)

- `installFromZip(SplFileInfo $zip): Plugin`
 - `enabled(): Collection<Plugin>`
 - `buildBlocksRegistry(): array` → per `window.__PLUGIN_BLOCKS__`
 - `adminAssets(): ['css'=>[], 'js'=>[]]`
 - `frontAssets(): string[]` (flat)
 - `frontendAssets(): string[]` (entry fallback)
 - `assetUrl(Plugin $p, string $rel): string`
 - `assetUrlVersioned(Plugin $p, string $rel): string`
-

12) Checklist per il rilascio del plugin

- [] `plugin.json` con `slug` univoco e `blocks[].type` che inizia con `plugin:`
 - [] `public/` con `view.js` (IIFE) e, se serve, `view.css` e `admin.js`
 - [] Nessun `type="module"`; lo script registra su `window.BuilderPlugins`
 - [] Classi CSS prefissate (niente clash con Bootstrap)
 - [] Bundle minificati; niente path assoluti hard-coded
 - [] Test: installa da ZIP, abilita, inserisci il blocco in una pagina, verifica front
-

13) Estensioni/sicurezza (consigli)

- Valida ulteriormente il manifest (es. che `blocks[].type` rispetti `^plugin:[a-z0-9-]+$`).
 - (Opz.) Tieni `enabled=false` dopo l'installazione e abilita a mano.
 - Evita di eseguire PHP proveniente dai plugin: il sistema è front-only.
 - Logga risultati e mtime per debug degli asset versionati.
-

Pronto! Con questa struttura puoi creare, installare e usare plugin nel CMS-R4 in modo consistente e predicibile. Se vuoi, posso anche generare uno **scheletro ZIP** di esempio per il tuo `r4-logos-carousel1`.